Lab3_N_Grams_ipynb-2

September 15, 2026

1 Ahmad Eyad Muhanna 2230006180

2 N-Grams

It's a probabilistic model that's trained on a corpus of text. Such a model is useful in many NLP applications including speech recognition, machine translation and predictive text input. An N-gram model is built by counting how often word sequences occur in corpus text and then estimating the probabilities.

2.1 Types of N-Grams

2.2 Use-cases of N-Grams

- Auto completion of sentences
- Auto spell check
- Voice-based personal assistant bots

2.3 Unigram

Unigram is an n-gram consisting of a single item of a sequence.

- It is commonly used to calculate the probability of finding a word in an article.

$$P(A) = \frac{\text{count of } A}{\text{count of all words}}$$

[1]:
```
import nltk
from nltk.util import ngrams
from nltk.tokenize import word_tokenize

string = "moses supposes his toeses are roses but moses supposes erroneously"

unigrams = ngrams(word_tokenize(string), 1)

for item in unigrams:
    print(item)
```

```
('moses',)
('supposes',)
('his',)
('toeses',)
('are',)
('roses',)
('but',)
('moses',)
('supposes',)
('erroneously',)
```

2.3.1 Calculating probabilities in unigram models

[2]:
```
### Calculate counts of each word

string = "moses supposes his toeses are roses but moses supposes erroneously"

unique_words = set(word_tokenize(string))

words_frq = []
for uniques in unique_words:
    frq = string.count(uniques)
    words_frq.append((uniques, frq))

words_frq
```

[2]:
```
[('erroneously', 1),
 ('are', 1),
 ('supposes', 2),
 ('but', 1),
 ('his', 1),
 ('moses', 2),
 ('toeses', 1),
 ('roses', 1)]
```

[3]:
```
### Calculate probabilities:

probab = []

for count in range(len(words_frq)):
    pro = words_frq[count][1]/len(word_tokenize(string))
    probab.append((words_frq[count][0],pro))

print('The probabilities are: ', probab)
```

```
The probabilities are:  [('erroneously', 0.1), ('are', 0.1), ('supposes', 0.2),
('but', 0.1), ('his', 0.1), ('moses', 0.2), ('toeses', 0.1), ('roses', 0.1)]
```

2.4 Bigram

Bigram an n-gram consisting of two items of a sequence - Two words that are used together to mean something specific

$$P(B \mid A) = \frac{\text{Count of } (AB)}{\text{Count of } (A)}$$

2.4.1 Method 1

[4]:
```
import nltk
from nltk.tokenize import word_tokenize

string = "moses supposes his toeses are roses but moses supposes erroneously"

bigrams = nltk.bigrams(word_tokenize(string))
for i in bigrams:
    print(i)
```

```
('moses', 'supposes')
('supposes', 'his')
('his', 'toeses')
('toeses', 'are')
('are', 'roses')
('roses', 'but')
('but', 'moses')
('moses', 'supposes')
('supposes', 'erroneously')
```

2.4.2 Method 2

[5]:
```
from nltk.util import ngrams
from nltk.tokenize import word_tokenize

string = "moses supposes his toeses are roses but moses supposes erroneously"

bigrams = ngrams(word_tokenize(string),2)

for i in bigrams:
    print(i)
```

```
('moses', 'supposes')
('supposes', 'his')
('his', 'toeses')
('toeses', 'are')
('are', 'roses')
('roses', 'but')
('but', 'moses')
```

```
('moses', 'supposes')
('supposes', 'erroneously')
```

3 Trigram

Trigram an n-gram consisting of three items of a sequence - Three words that are used together to mean something specific

3.0.1 Method 1

[6]:
```
import nltk
from nltk.tokenize import word_tokenize

string = "moses supposes his toeses are roses but moses supposes erroneously"

trigrams = nltk.trigrams(word_tokenize(string))

for i in trigrams:
    print(i)
```

```
('moses', 'supposes', 'his')
('supposes', 'his', 'toeses')
('his', 'toeses', 'are')
('toeses', 'are', 'roses')
('are', 'roses', 'but')
('roses', 'but', 'moses')
('but', 'moses', 'supposes')
('moses', 'supposes', 'erroneously')
```

3.0.2 Method 2

[7]:
```
from nltk.util import ngrams
from nltk.tokenize import word_tokenize

string = "moses supposes his toeses are roses but moses supposes erroneously"

trigrams = ngrams(word_tokenize(string),3)

for i in trigrams:
    print(i)
```

```
('moses', 'supposes', 'his')
('supposes', 'his', 'toeses')
('his', 'toeses', 'are')
('toeses', 'are', 'roses')
('are', 'roses', 'but')
('roses', 'but', 'moses')
```

```
('but', 'moses', 'supposes')
('moses', 'supposes', 'erroneously')
```

3.1 What is padding in NLP?

Padding is a technique used to ensure that all input sequences have the same length. This is necessary because neural networks require inputs that have the same shape and size. However, when we pre-process and use texts as inputs for our model, not all sentences have the same length. In other words, some sentences are longer or shorter than others. We need to have inputs with the same size, and this is where padding comes in.

Padding is a special form of masking where the masked steps are at the start or the end of a sequence. Padding comes from the need to encode sequence data into contiguous batches: in order to make all sequences in a batch fit a given standard length. The padding is added before splitting the sentence.

n-1 padding is added.

[8]:
```
from nltk.util import ngrams
from nltk.tokenize import word_tokenize


string = "moses supposes his toeses are roses but moses supposes erroneously"

print('------------------- Padding in Bigrams ------------------------------')
list(ngrams(word_tokenize(string), pad_left=True, left_pad_symbol="<s>",
                                pad_right=True, right_pad_symbol="</s>", n=2))
```

```
------------------- Padding in Bigrams ------------------------------
```

[8]:
```
[('<s>', 'moses'),
 ('moses', 'supposes'),
 ('supposes', 'his'),
 ('his', 'toeses'),
 ('toeses', 'are'),
 ('are', 'roses'),
 ('roses', 'but'),
 ('but', 'moses'),
 ('moses', 'supposes'),
 ('supposes', 'erroneously'),
 ('erroneously', '</s>')]
```

[9]:
```
print('------------------- Padding in Trigrams ------------------------------')
list(ngrams(word_tokenize(string), pad_left=True, left_pad_symbol="<s>",
                                pad_right=True, right_pad_symbol="</s>", n=3))
```

```
------------------- Padding in Trigrams ------------------------------
```

[9]:
```
[('<s>', '<s>', 'moses'),
 ('<s>', 'moses', 'supposes'),
```

```
 ('moses', 'supposes', 'his'),
 ('supposes', 'his', 'toeses'),
 ('his', 'toeses', 'are'),
 ('toeses', 'are', 'roses'),
 ('are', 'roses', 'but'),
 ('roses', 'but', 'moses'),
 ('but', 'moses', 'supposes'),
 ('moses', 'supposes', 'erroneously'),
 ('supposes', 'erroneously', '</s>'),
 ('erroneously', '</s>', '</s>')]
```

Alternatively, We can get rid of the parameters by using > pad_both_ends method in NLTK

Note the n argument, that tells the function we need padding for bigrams.

[10]:
```
from nltk.lm.preprocessing import pad_both_ends
from nltk.util import ngrams
from nltk.tokenize import word_tokenize

string = "moses supposes his toeses are roses but moses supposes erroneously"

list(ngrams(pad_both_ends(word_tokenize(string), n=2), n=2))
```

[10]:
```
[('<s>', 'moses'),
 ('moses', 'supposes'),
 ('supposes', 'his'),
 ('his', 'toeses'),
 ('toeses', 'are'),
 ('are', 'roses'),
 ('roses', 'but'),
 ('but', 'moses'),
 ('moses', 'supposes'),
 ('supposes', 'erroneously'),
 ('erroneously', '</s>')]
```

3.1.1 Training the Maximum Likelihood Model

Maximum likelihood estimation estimates the model parameters such that the probability is maximized.

- Get the requisite n_grams frequency counts from a corpus.
- Normalize them to a 0-1 range.
- Build the vocabulary (of unique words): to create this vocabulary we need to pad our sentences (just like for counting ngrams) and then combine the sentences into one flat stream of words.

To do the above steps, we need to create an Ngram and add the padding then flatten the list. All these steps are requested for pre-processing to train the MLE model. However, we can use padded_everygram_pipeline() in NLTK to do all the steps in one call.

The padded_everygram_pipeline() will return:
1- **ngrams_gen:** Each sentence becomes a list of n-grams with padding.
2- **vocab_gen:** A flat stream of all tokens, with start and end symbols included

References:

https://www.nltk.org/api/nltk.lm.html

https://www.kaggle.com/code/alvations/n-gram-language-model-with-nltk

[11]:
```
from nltk.lm.preprocessing import padded_everygram_pipeline
from nltk.util import ngrams
from nltk.tokenize import word_tokenize
from nltk.lm import smoothing

string = "moses supposes his toeses are roses but moses supposes erroneously"

#Create Bigram Language Model
train, vocab = padded_everygram_pipeline(2, [word_tokenize(string)])
```

[12]:
```
from nltk.lm import MLE
lm = MLE(2)
lm.fit(train, vocab) #Train the Maximum Likelihood Model
```

3.1.2 Calculating Probabilities

Here's how you get the score for a word given some preceding context. For example we want to know what is the chance that "b" is preceded by "a".

[13]:
```
print(lm.score('supposes', ['moses'])) #Calculate Bigram Probabilities for␣
↪P('moses'|'supposes')
print(lm.score('moses', ['supposes'])) #Calculate Bigram Probabilities for␣
↪P('supposes'|'moses') - Returns 0 because unseen
```

```
1.0
0.0
```

3.1.3 Evaluating n_grams models using preplexity

Preplexity is a measurement of how well a probability model (like a language model) predicts a sample.

It's the probability of the test set normalized by the number of words.

A `lower` perplexity means:

- The model is less "perplexed" by the data.
- The model assigns higher probabilities to the actual words in the test set.
- Therefore, better performance.

A `higher perplexity` = more uncertainty = worse model.

[14]:
```
test = [('moses', 'supposes'), ('toeses','are')]
print(lm.perplexity(test)) #Perplexity for bigram model
```

```
1.0
```

3.2 Task: Generating tweets using n_grams

In this Task, we will build a MLE model with n-gram to generate tweets.

The agenda is: 1. load the data: https://www.kaggle.com/datasets/adizafar/large-random-tweets-from-pakistan

2. pre-processing the data:

- Remove hashtags
- Remove RT
- Remove any website
- Remove any mentions
- Remove emojis

3. build MLE model with n-gram (Bi-gram)
4. evaluate the model
5. calculate the probability of the bigram: (pakistan is)
6. calculate the preplexity of the word: (pakistan)

before we start we need to install library to handle the emojis in tweets > pip install emoji

[15]:
```
%pip install emoji
```

```
Collecting emoji
  Downloading emoji-2.15.0-py3-none-any.whl.metadata (5.7 kB)
Downloading emoji-2.15.0-py3-none-any.whl (608 kB)
   ---------------------------------------- 0.0/608.4 kB ? eta -:--:--
   ---------------------------------------- 0.0/608.4 kB ? eta -:--:--
   ---------------- ---------------------- 262.1/608.4 kB ? eta -:--:--
   ---------------------------------------- 608.4/608.4 kB 2.4 MB/s  0:00:00
Installing collected packages: emoji
Successfully installed emoji-2.15.0
Note: you may need to restart the kernel to use updated packages.


[notice] A new release of pip is available: 26.0.1 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

[17]:
```
import kagglehub

# Download latest version
path = kagglehub.dataset_download("adizafar/large-random-tweets-from-pakistan")

print("Path to dataset files:", path)
```

```
c:\Users\user\AppData\Local\Programs\Python\Python313\Lib\site-
packages\tqdm\auto.py:21: TqdmWarning: IProgress not found. Please update
jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm

Downloading to C:\Users\user\.cache\kagglehub\datasets\adizafar\large-random-
tweets-from-pakistan\2.archive…

100%|      | 11.2M/11.2M [00:02<00:00, 5.80MB/s]

Extracting files…


Path to dataset files: C:\Users\user\.cache\kagglehub\datasets\adizafar\large-
random-tweets-from-pakistan\versions\2
```

[18]:
```
import re
import random
import emoji
import pandas as pd
import nltk
from nltk.util import bigrams
from nltk.lm.preprocessing import padded_everygram_pipeline
from nltk.lm import MLE

nltk.download('punkt', quiet=True)
nltk.download('punkt_tab', quiet=True)
```

[18]:
```
True
```

[20]:
```
df = pd.read_csv("Random Tweets from Pakistan- Cleaned- Anonymous.csv",␣
↪encoding="utf-8", encoding_errors="replace", low_memory=False)
print(df.shape)
df.head()
```

```
(202202, 7)
```

[20]:

	Unnamed: 0	created_at_tweet	
0	0	Wed Jul 03 07:20:15 +0000 2019	
1	1	Fri Jul 02 02:50:53 +0000 2021	
2	2	Fri Jul 02 12:12:53 +0000 2021	
3	3	Tue Oct 15 22:29:16 +0000 2019	
4	4	Wed May 13 04:34:31 +0000 2020	

		full_text	retweet_count	\
0	…		226	
1	Happy birthday to my brother n boss , May you …		54	
2			50	
3	`suspicious °jikook au jimin'in yaşadığı kasab…		522	

```
4  Speaking of @SpiderMan…    https://t.co/uGJ…            74

  favorite_count reply_count               location
0           1097         127       Punjab, Pakistan
1            273           9       Punjab, Pakistan
2            131          13       Punjab, Pakistan
3           2134          86      İstanbul, Türkiye
4            535          11  Follow the ATP Tour
```

[21]:
```
text_col = "full_text"
tweets = df[text_col].dropna().astype(str).tolist()
print(f"Total tweets: {len(tweets)}")
```

```
Total tweets: 202151
```

[23]:
```
# getting rid of urdu text, because the model is english based
tweets = [t for t in tweets if sum(c.isascii() for c in t) / max(len(t), 1) > 0.
 ↪85]
print(f"English tweets: {len(tweets)}")
```

```
English tweets: 158519
```

[24]:
```
def clean_tweet(text):
    text = re.sub(r'^RT\s+', '', text)              # remove RT
    text = re.sub(r'RT\s*:', '', text)
    text = re.sub(r'http\S+|www\.\S+', '', text)     # remove URLs
    text = re.sub(r'@\w+', '', text)                 # remove mentions
    text = re.sub(r'#\w+', '', text)                 # remove hashtags
    text = emoji.replace_emoji(text, replace='')     # remove emojis
    text = re.sub(r'\s+', ' ', text).strip()
    return text.lower()

cleaned_tweets = [clean_tweet(t) for t in tweets]
cleaned_tweets = [t for t in cleaned_tweets if t]  # drop empties

print("Before:", tweets[0])
print("After :", cleaned_tweets[0])
```

```
Before: Happy birthday to my brother n boss , May you have many many more  ,
you are Gem  #HappyBirthdayAtifRauf
After : happy birthday to my brother n boss , may you have many many more, you
are gem
```

[25]:
```
tokenized_tweets = [t.split() for t in cleaned_tweets]
tokenized_tweets[:2]
```

[25]:
```
[['happy',
  'birthday',
  'to',
```

```
 'my',
 'brother',
 'n',
 'boss',
 ',',
 'may',
 'you',
 'have',
 'many',
 'many',
 'more,',
 'you',
 'are',
 'gem'],
['`suspicious',
 '°jikook',
 'au',
 "jimin'in",
 'yaşadığı',
 'kasabada',
 'aniden',
 'başlayan',
 'cinayetler,',
 'onun',
 'kendisini',
 'sürekli',
 'takip',
 'eden',
 'yeni',
 'komşusu',
 "jungkook'tan",
 'şüphelenmesine',
 'neden',
 'olur']]
```

[26]:
```
n = 2
train_data, padded_vocab = padded_everygram_pipeline(n, tokenized_tweets)

model = MLE(n)
model.fit(train_data, padded_vocab)

print(f"Vocabulary size: {len(model.vocab)}")
```

```
Vocabulary size: 140358
```

[27]:
```
def generate_tweet(model, num_words=15, seed=None):
    if seed is not None:
        random.seed(seed)
```

```
    content = model.generate(num_words, text_seed=['<s>'])
    words = [w for w in content if w not in ('<s>', '</s>')]
    return ' '.join(words)

for i in range(3):
    print(generate_tweet(model, num_words=15, seed=i))
```

```
overwhelming tourism, the floor flat as they arrange a week celebration. being
downloaded in
thank you just 40 lac for planning a hard in azad jammu &amp;amp; .
: that's the ambassador to providing economical crisis and sea mil gaye hai to
bring
```

[28]:
```
prob = model.score('is', ['pakistan'])
print(f"P('is' | 'pakistan') = {prob}")
```

```
P('is' | 'pakistan') = 0.06581762608252674
```

[31]:
```
test_ngrams = list(bigrams(
    ['pakistan'],
    pad_left=True, pad_right=True,
    left_pad_symbol='<s>', right_pad_symbol='</s>'
))
pp = model.perplexity(test_ngrams)
print(f"Perplexity of 'pakistan' = {pp}")
```

```
Perplexity of 'pakistan' = 47.585823313042205
```